Mašininio mokymosi operacijų sistemos (MLOps)

Praktinės užduotys — 1–3 savaitės

Laboratoriniai darbai

doc. dr. Linas Petkevičius, part. doc. Jonas Matuzas, Marius Šomka,
lekt. Povilas Kvedaras, lekt. Dmitrij Nikolajev,
lekt. Paulius Milmantas, Lukas Valatka

2026 m. rudens semestras

Pagrindinė studentų praktikos medžiaga.
Šiame leidinyje — projekto taisyklės ir 1–3 savaičių laboratoriniai darbai.
3 savaitė: CPU ir ONNX vaizdai, API, Trivy ir GHCR.
Ubuntu paruošimas — šio PDF priede, p. 35.
3 savaitės instrukcija atnaujinta 2026-09-17 pagal studentų repozitoriją.
Tolesnių savaičių užduotys skelbiamos semestro eigoje.

Turinys

1 Praktinių užsiėmimų apžvalga **3**
1.1 Struktūra ir valandų paskirstymas . . . 3
1.2 Vertinimo struktūra . . . 3
1.3 Infrastruktūros gairės . . . 3
1.4 16 modulių praktikos apžvalga (`topics.md`) . . . 3

2 Semestro projektas **4**
2.1 Projekto koncepcija . . . 4
2.2 Projekto etapai . . . 4
2.3 Komandinio projekto reikalavimai . . . 5
2.4 Komandinio projekto vertinimo kriterijai . . . 5

3 Individualių užduočių sistema **5**
3.1 Bendros taisyklės . . . 6
3.2 U1 — Kubernetes klasteris ir MLflow serveris (iki 8 sav.) . . . 6
3.3 U2 — Modelio mokymas su MLflow ir hostingas klasteryje (iki 11 sav.) . . . 8
3.4 Lokalus Kubernetes klasteris ir GPU . . . 9
3.5 Užduočių kalendorius . . . 10

4 1 savaitė — praktika, tutorialai ir užduotys **10**
4.0.1 Seminaras (2 val.) — detalus scenarijus . . . 10
4.0.2 Laboratorinis darbas (3 val.) — semestro projekto startas . . . 12
4.0.3 Vertinimo kriterijai (1 savaitė, iki 0,2 balo) . . . 15
4.0.4 Savarankiškas darbas (5 val.) . . . 15
4.0.5 Tutorial 1.0 — Git repo sukūrimas nuo nulio . . . 16
4.0.6 Seminaro darbo lapas (1 savaitė) . . . 16
4.0.7 Pilnas pavyzdys: užpildytas reikalavimų dokumentas . . . 17
4.0.8 Papildomos praktinės užduotys (1 savaitė) . . . 17

5 2 savaitė — praktika, tutorialai ir užduotys **17**
5.0.1 Apibendrinimas (10 min.) — atkartojamumo katastrofa . . . 17
5.0.2 Laboratorinis darbas — MLflow + DVC pipeline . . . 18
5.0.3 Savęs vertinimo kriterijai — ką turėtumėte turėti po 2 savaitės . . . 24
5.0.4 Savarankiškas darbas (5 val.) — išplėstinis kelias . . . 25
5.0.5 Tutorial 2.1 — pilnas `prepare_data.py` . . . 26
5.0.6 Tutorial 2.2 — žingsnis po žingsnio pipeline paleidimas . . . 26
5.0.7 Tutorial 2.3 — `.dvc` pointer failo anatomija . . . 26
5.0.8 Tutorial 2.4 — prisijungimas prie dėstytojo serverio ("at scale" akimirka) 27
5.0.9 Tutorial 2.5 — savo duomenys: Hugging Face → parquet → DVC . . . 27
5.0.10 Tutorial 2.6 (išplėstinis kelias) — LLM fine-tuning su W&B monitoringu 27
5.0.11 Tutorial 2.7 (išplėstinis kelias) — tikri DVC remote'ai: SSH, MinIO/S3, GCS . . . 28
5.0.12 Troubleshooting — dažnos problemos (2 savaitė) . . . 29
5.0.13 Papildomos praktinės užduotys (2 savaitė) . . . 29

6 3 savaitė — Konteinerizacija: praktika **30**
6.0.1 0. Pasiruošimas prieš užsiėmimą . . . 30
6.0.2 1. CPU ir ONNX: build, API ir saugumas . . . 30
6.0.3 2. Publikavimas į GHCR . . . 31
6.0.4 3. Išsaugoti darbą . . . 32
6.0.5 GPU užduotis: tik NVIDIA Linux serveryje . . . 32
6.0.6 Papildomos užduotys . . . 32
6.0.7 Atsiskaitymas (iki 0,2 balo) . . . 33

7 Savarankiško darbo valandų suvestinė **33**

8 Atsiskaitymo kalendorius **34**

A Ubuntu paruošimas **35**
A.1 Bendri įrankiai . . . 35
A.2 Docker Engine ir Buildx . . . 35
A.3 Trivy . . . 36
A.4 GitHub CLI (gh) . . . 36
A.5 Grįžimas į praktiką . . . 36

1 Praktinių užsiėmimų apžvalga

1.1 Struktūra ir valandų paskirstymas

Kiekvieną savaitę (1–15) vyksta:

- **2 val. seminaras** — temų aptarimas, atvejų analizė, užduočių planavimas, tarpiniai rezultatai.
- **3 val. laboratorinis darbas** — praktinis užsiėmimas su konkrečiu deliverable.

16 savaitė skirta **projektų pristatymams** (3 val. seminaras).

Veikla	Val./sav.	Savaitės	Iš viso
Seminarai	2	15	30
Laboratoriniai darbai	3	15	45
Projektų pristatymai	3	1	3
Kontaktinis darbas			**48***
Savarankiškas darbas			**82**

**Kartu su 15 paskaitų valandomis ir 3 konsultacijomis.*

1.2 Vertinimo struktūra

Komponentas	Aprašymas	Svoris
Individuali užduotis	Atsiskaitoma per laboratorinius darbus (iki 3 balai)	30 %
Komandinis projektas	Pristatymas ir gynimas semestro pabaigoje (iki 4 balai)	40 %
Egzaminas	Raštu, pagal sesijos tvarkaraštį (iki 3 balai)	30 %

Egzamino sąlygos: galima laikyti tik apginus komandinį projektą ($\geq$2 balai) ir atsiskaičius individualią užduotį ($\geq$1,5 balo).

1.3 Infrastruktūros gairės

- **GCP** — Terraform diegimai, Cloud Storage, GKE (be GPU), Vertex AI Endpoints, Artifact Registry. Studentų kreditai eksperimentams debesų aplinkoje.
- **Universiteto HPC** — GPU resursai: mokymas, inferencija (vLLM, TensorRT), paskirstytas mokymas (DDP/Horovod), profiliavimas.
- **GitHub** — Visi deliverable'ai commit'inami į komandinį repo su aiškiais tag'ais (`v1.0`, `v1.1`, `week-03` ir pan.).

1.4 16 modulių praktikos apžvalga (`topics.md`)

Sav.	Modulis	Laboratorinis deliverable
1	MLOps, AIOps, RE	Pradinis `requirements.md` (FR+NFR+KPI); tag `v1.0`
2	Atkartojamumas	MLflow server + DVC pipeline; `reproduce.sh`; `v1.1`
3	Docker	3 vaizdai (CPU/GPU/ONNX); GHCR; Trivy scan
4	Cloud-native	Terraform GCP aplinka; managed endpoint
5	Kubernetes	Multi-node GPU cluster; PyTorch DDP / Horovod
6	CI/CD	GitHub → Terraform → ArgoCD → KServe; auto-promote

7	Model serving	7B LLM vLLM; autoscaling; 4-bit kvantizacija; benchmark
8	GPU architektūra	GPU profiliavimas; $\geq 2\times$ speedup ataskaita
9	Orkestravimas	Ray pipeline (data→deploy); human approval
10	DI agentai	Self-healing agentas: drift→retrain→deploy→Slack
11	Observability	Prometheus/Grafana/Loki/Evidently; dashboard'ai + alertai
12	Scaling	Locust 1→50 replikų; spot+Karpenter; >60 % cost cut
13	Saugumas	Gatekeeper, secrets, encrypted PVC, container scanning
14	Edge, FL	TFLite/ONNX Mobile edge deploy; Flower FL simuliacija
15	Testavimas	Great Expectations + Evidently + E2E; traceability matrix
16	Governance + demo	Capstone Live Demo Day (15 min) + governance ataskaita

Šiame leidinyje: 1–3 savaičių praktika. Tolesnių savaičių tutorialai skelbiami semestro eigoje; viso semestro deliverable'ų žemėlapis — lentelėje aukščiau.

2 Semestro projektas

2.1 Projekto koncepcija

Viso semestro metu studentai (grupėse po 2–3 asmenis) kuria **produkcinę DI sistemą**, integruojančią kurso temas: duomenų paruošimas → mokymas → registras → hostingas → stebėsena. Temą (modelį ir duomenis) komanda **pasirenka pati ir suderina su lektoriumi**; kurso pavyzdinis fraud detection modelis komandiniam projektui nenaudojamas. Kiekvieno laboratorinio darbo rezultatas prisideda prie projekto. Galutinis rezultatas — pilnai veikianti sistema lokaliame Kubernetes klasteryje (papildomai — GCP ir/ar HPC) su stebėsena, testavimu ir governance ataskaita.

2.2 Projekto etapai

1. **1–2 sav.** — Reikalavimų dokumentas (v1.0 → v1.1); repo struktūra; modelio ir duomenų pasirinkimas; MLflow + DVC setup.
2. **3–5 sav.** — Docker, GCP/Terraform, Kubernetes (konteinerizacija + debesys); lokalus klasteris.
3. **6–9 sav.** — CI/CD, model serving, GPU optimizavimas, workflow orkestravimas; komandos temos suderinimas (9 sav.).
4. **10–13 sav.** — DI agentas, observability, scaling, saugumas/compliance.
5. **14–15 sav.** — Edge MLOps, federated learning, testavimas, traceability matrix; projekto pateikimas (tag `team-final`, 15 sav.).
6. **16 sav.** — Responsible AI governance ataskaita + Capstone Live Demo Day.

Individualios užduotys U1 (8 sav.) ir U2 (11 sav.) — žr. kitą skyrių; jų rezultatai (klasteris, MLflow serveris, pipeline'as, serving'as) tiesiogiai pernaudojami komandiniame projekte.

2.3 Komandinio projekto reikalavimai

- **Problema ir duomenys.** Tema pasirenkama komandos ir suderinama su lektoriumi iki 9 sav.; duomenys pasirinktinai (rekomenduojami atviri Hugging Face rinkiniai), legalūs, be asmens duomenų arba anonimizuoti. `docs/requirements.md` su pilna versijų istorija ir atsekamumo matrica (NFR → testas → įrodymas).
- **Duomenų ir mokymo pipeline'as.** DVC pipeline'as + MLflow tracking komandos MLflow serveryje (registras, alias'ai); orkestravimas — bent DVC + CI, rekomenduojama Ray/Prefect DAG su žmogaus patvirtinimu (HITL).
- **Hostingas.** Inferencijos servisas klasteryje, CI/CD su kokybės vartu, versijuoti vaizdai, rollback.
- **Stebėsena.** Prometheus + Grafana (vėlinimas, klaidos, pralaidumas) + Evidently dreifo ataskaita; ≥2 alert'ai; `docs/runbook.md`.
- **Mastelis, saugumas, testavimas** — bent po vieną elementą iš kiekvienos temos: HPA + apkrovos testas; non-root vaizdai + Trivy + Secret'ai (ne Git'e) + ≥1 policy; duomenų kontrakto, modelio kokybės ir E2E testai CI.
- **Governance.** Model card (≥8 skyriai) + trumpa rizikų / EU AI Act klasifikacija.
- **Ataskaita** (iki 4 psl.): metodologija, architektūros diagrama, rezultatai, išvados.
- **Gynimas** (16 sav.): 15 min. pristatymas + 5 min. klausimai; live demo produkcinėje aplinkoje.
- Vertinama iki 4 balų; projektas laikomas apgintu, jei surinkta ≥2 balai.

Išplėstinė dalis (papildomi balai iki maksimumo): GPU mokymas (lokaliai arba HPC), self-healing agentas, edge/FL variantas, kaštų analizė GCP.

2.4 Komandinio projekto vertinimo kriterijai

Kriterijus	**Balai**
Reikalavimai, architektūra, atsekamumo matrica	_____
Duomenų + mokymo pipeline'as (DVC, MLflow, registras, orkestravimas)	_____
Hostingas klasteryje + CI/CD + rollback	_____
Stebėsena, dreifas, alert'ai, runbook	_____
Mastelis / saugumas / testavimas	_____
Ataskaita + model card	_____
Gynimas (demo, klausimai)	_____
Iš viso	**4**

3 Individualių užduočių sistema

Individuali dalis (**3 balai**, 30 %) susideda iš **dviejų užduočių**, kurios kartu sudaro studento "mini MLOps platformą": U1 — infrastruktūra (klasteris, savas MLflow serveris, atkartojamas pipeline'as), U2 — modelio mokymas, hostingas ir automatizavimas toje infrastruktūroje. Savaitiniai laboratoriniai darbai (iki 0,2 balo, žr. kiekvienos savaitės vertinimo lenteles) vertinami kaip **papildomi balai**.

Užduotis	Pateikimas	Balai	Reikalingos paskaitos
U1. Kubernetes klasteris + MLflow serveris	8 sav. (2026-10-21)	_____ iš 3	1–6 (branduolys — iki 6 sav.)
U2. Modelio mokymas su MLflow ir hostingas klasteryje	11 sav. (2026-11-11)	_____ iš 3	2, 5–7 (branduolys); 8–9 (išplėstinė dalis)

Egzaminą laikyti galima, jei apgintas komandinis projektas ($\geq$2 balai) ir atsiskaitytos individualios užduotys (U1 + U2 $\geq$1,5 balo).

3.1 Bendros taisyklės

1. **Repozitorija.** Kiekvienam studentui — atskira GitHub repozitorija (šablonas pagal `tutorial/fraud-detection` struktūrą); pateikimai žymimi Git tag'ais `u1-final`, `u2-final`; `docs/requirements.md` versijuojamas (v1.0 → …).

2. **Pateikimas ir gynimas.** Užduotys **pateikiamos iki nurodytos datos**; **gynimas vyksta gyvai per laboratorinį darbą** (5–10 min. demonstracija, dėstytojo klausimai, live edit), prireikus — vėlesniame užsiėmime. Be demonstracijos užduotis nevertinama.

3. **Modelis ir duomenys.** Kiekvienas studentas **pasirenka savo** modelį ir duomenų rinkinį (rekomenduojami atviri Hugging Face rinkiniai); kurso fraud detection pavyzdys neleidžiamas. Užtenka mažo tabulinio / tekstinio modelio (mokymas CPU per kelias minutes) — vertinamas srautas aplink modelį, ne jo kokybė.

4. **Referencinis kodas ir DI asistentai.** Leidžiama naudoti `tutorial/fraud-detection` kodą, laboratorinių darbų medžiagą ir DI asistentus, bet studentas privalo gebėti paaiškinti ir modifikuoti kiekvieną pateiktą failą. Jei studentas nepajėgia paaiškinti savo sprendimo, dėstytojas gali mažinti balą arba neužskaityti darbo.

5. **Aplinka.** Viskas veikia studento kompiuteryje su Docker ir lokaliu Kubernetes klasteriu (žr. 3.4 skyrių). GPU **neprivalomas**: turintiems GPU (savo kompiuteryje arba HPC) rekomenduojama jį panaudoti išplėstinėje dalyje, tačiau balas nemažinamas, jei GPU nenaudojamas. GCP kreditai — papildoma galimybė.

6. **MLflow serveris.** Bendro kurso MLflow serverio nėra — kiekvienas studentas hostina **savo** MLflow Tracking Server savo klasteryje (U1) ir naudoja jį U2 bei komandiniame projekte.

7. **Branduolys ir išplėstinė dalis.** Branduolys būtinas užskaitymui; išplėstinė dalis — papildomi balai iki užduoties maksimumo.

8. **README** kiekvienoje repozitorijoje: kaip paleisti nuo nulio, portai, prisijungimai, kur žiūrėti rezultatus.

3.2 U1 — Kubernetes klasteris ir MLflow serveris (iki 8 sav.)

U1: repo + reikalavimai, MLflow + DVC pipeline'as, Docker vaizdas GHCR, lokalus klasteris su savu MLflow serveriu

Tikslas: turėti savo "mini MLOps platformą", į kurios MLflow serverį klasteryje log'inami eksperimentai. Branduoliui užtenka 1–6 sav. medžiagos (CI/CD perkeltas į U2); 7–8 sav. medžiaga nereikalinga.

Reikalavimai (branduolys).

U1-R1 Repozitorija su struktūra (`src/`, `docker/`, `k8s/`, `docs/`), README, `docs/requirements.md` ≥ v1.1 (≥3 FR, ≥5 NFR su KPI, tarp jų reprodukcijos NFR) savo pasirinktai problemai.

U1-R2 Atkartojamas pipeline'as: `dvc.yaml` (≥3 stage'ai: prepare → train → evaluate), `params.yaml`, `dvc.lock` commit'intas, DVC remote (lokalus katalogas, SSH, MinIO arba GCS), ≥3 MLflow run'ai su skirtingais hiperparametrais, `reproduce` scenarijus, atkuriantis pasirinktą run'ą per <2 min.

U1-R3 Docker vaizdas (mokymo arba inferencijos) iš `Dockerfile` repozitorijoje, publikuotas GHCR su tag'u `sha-<commit>`; vaizdo dydis ir Trivy skenavimo rezultatas užfiksuoti `docs/`.

U1-R4 Lokalus Kubernetes klasteris (konfigūracija repozitorijoje, pvz. `k8s/kind-config.yaml`); jame atskiras Namespace ir **MLflow Tracking Server** (Deployment + Service + PVC; rekomenduojama PostgreSQL + artefaktų PVC), pasiekiamas iš studento kompiuterio (port-forward arba NodePort); mokymo scenarijus log'ina į klasterio MLflow.

Išplėstinė dalis.

U1-E1 MLflow serveris su autentifikacija (`--app-name basic-auth`), ≥2 vartotojai / rolės.

U1-E2 Mokymas kaip Kubernetes Job klasteryje, log'inantis į klasterio MLflow.

U1-E3 Klasteris ir jo turinys kaip kodas: Terraform (kind / minikube / k3d provider'is) + Helm chart'as arba Kustomize overlay'ai MLflow serveriui.

U1-E4 DVC remote debesyje (GCS per Terraform).

U1-E5 GPU lokaliame klasteryje: `nvidia.com/gpu` node capacity ir Pod'as su `nvidia-smi`.

Užduoties rezultatas (deliverable)

- **D1:** repo URL + tag `u1-final`; **D2:** `docs/requirements.md` ≥ v1.1.
- **D3:** GHCR vaizdo nuoroda; **D4:** `k8s/` manifestai + klasterio konfigūracija.
- **D5:** MLflow UI su ≥3 run'ais klasterio serveryje (demonstracija).
- **D6:** `docs/u1-report.md` (1 psl.: kas padaryta, kas neveikė, kiek laiko užtruko).

Vertinimo kriterijai (U1).

Kriterijus	Balai	Minimalus slenkstis
Repo, README, `requirements.md` su KPI	_____	≥5 NFR su KPI
MLflow + DVC pipeline'as, `reproduce` <2 min.	_____	`dvc repro` veikia; ≥3 run'ai
Docker vaizdas GHCR, skenuotas	_____	vaizdas pasileidžia
Lokalus klasteris + MLflow serveris jame	_____	run'as iš kompiuterio matomas klasterio UI
Gynimas (demo, klausimai, live edit)	_____	atsako į dėstytojo klausimus
Išplėstinė dalis (E1–E5)	iki _____	neviršijant U1 maksimumo
U1 maksimumas / užskaityta nuo	_____ / _____	

Gynimo klausimų pavyzdžiai: kur fiziškai yra MLflow artefaktai klasteryje ir kas nutiks perkūrus Pod'ą? Kas pasikeis `dvc.lock`, pakeitus `params.yaml`? Kodėl tag'as `latest` — bloga praktika? Paaiškinkite kiekvieną `kubectl get all -n <ns>` objektą.

3.3 U2 — Modelio mokymas su MLflow ir hostingas klasteryje (iki 11 sav.)

U2: mokymas su pilnu eksperimentų sekimu, registras, mokymo Job ir inferencijos servisas klasteryje, CI/CD su kokybės vartu

Tikslas: pilnas kelias nuo duomenų iki veikiančio inferencijos API klasteryje. Branduolys — 2, 5, 6, 7 sav. medžiaga; 8 sav. (GPU) ir 9 sav. (orkestravimas) — tik išplėstinė dalis.

Reikalavimai (branduolys).

U2-R1 Mokymas su MLflow "pagal visas taisykles": parametrai, metrikos per žingsnius (`step`), sistemos metrikos, duomenų rinkinio registravimas (`log_input` su digest'u), seed'as ir git commit tag'ai, aplinkos fiksavimas prie modelio, **įdėtiniai run'ai** hiperparametrų paieškai ($\geq$1 paieška su $\geq$5 bandymais — Optuna, Hydra `--multirun` arba sklearn autolog).

U2-R2 Modelis kaip artefaktas: `log_model(name=…)`, registruotas Model Registry su alias'u (`@champion`); inferencijos servisas modelį ima iš registro pagal alias'ą, ne iš failo.

U2-R3 Mokymo Job klasteryje (CPU), log'inantis į U1 MLflow serverį; parametrai per ConfigMap, prisijungimai per Secret.

U2-R4 Inferencijos API (FastAPI / `mlflow models serve` / KServe) konteineryje: Deployment + Service, `/health`, `/predict`; bazinis apkrovos testas (pvz. 100 užklausų, p95 vėlinimas užfiksuotas `docs/`).

U2-R5 CI (perkelta iš U1): GitHub Actions — lint + unit testai + vaizdo build ir publikavimas į GHCR.

U2-R6 CD su kokybės vartu ir rollback: testai $\rightarrow$ `dvc repro` (arba mokymo Job) $\rightarrow$ kokybės vartas (metrika $\geq$ slenkstis iš `metrics.json`) $\rightarrow$ vaizdo build/push $\rightarrow$ manifesto atnaujinimas; parodytas rollback į ankstesnę versiją ir **raudonas** paleidimas (blogas modelis nepraeina).

U2-R7 `docs/requirements.md` $\geq$ v1.6 su inferencijos NFR (p95 vėlinimas, pralaidumas, modelio dydis) ir išmatuotomis reikšmėmis.

Išplėstinė dalis (pasirenkama pagal turimus išteklius; rekomenduojama, balas nemažinamas, jei nedaroma).

U2-E1 GPU: mokymas lokaliame klasteryje arba HPC (AMP, profiliavimas); HPC run'ai perkeliami į klasterio MLflow (`mlflow-export-import`).

U2-E2 Orkestravimas: prepare $\rightarrow$ train $\rightarrow$ eval $\rightarrow$ register $\rightarrow$ deploy kaip Ray / Prefect / Airflow DAG su sąlyginiu žingsniu ir HITL patvirtinimu (Prefect — lengviausias išteklių prasme).

U2-E3 HPA inferencijos Deployment'ui ir apkrovos testas su Locust.

U2-E4 ONNX eksportas, mažesnis inferencijos vaizdas, vėlinimo palyginimas.

U2-E5 ArgoCD / canary diegimas.

Užduoties rezultatas (deliverable)

- **D1:** tag `u2-final`; **D2:** MLflow UI — HPO run'ų medis, registruotas modelis su alias'u.
- **D3:** `k8s/` Job + Deployment + Service manifestai.
- **D4:** CI/CD workflow su kokybės vartu (žalias **ir** raudonas paleidimas).
- **D5:** `docs/u2-report.md` ($\leq$2 psl.: architektūra, matavimai, rollback įrodymas); **D6:** `docs/requirements.md` $\geq$ v1.6.

Vertinimo kriterijai (U2).

Kriterijus	Balai	Minimalus slenkstis
Mokymas su pilnu tracking'u + HPO	_____	$\geq$5 bandymai; sistemos metrikos matomos
Registras + alias; servisas ima modelį iš registro	_____	`models:/<name>@champion`
Mokymo Job klasteryje	_____	Job baigiasi `Completed`; run'as UI
Inferencijos servisas klasteryje + matavimai	_____	`/predict` atsako; p95 užfiksuotas
CI (lint, testai, build/push)	_____	workflow žalias
CD: kokybės vartas + rollback	_____	raudonas paleidimas blokuoja deploy
Gynimas	_____	live edit: pakeisti slenkstį / parametrą
Išplėstinė dalis (E1–E5)	iki _____	neviršijant U2 maksimumo
U2 maksimumas / užskaityta nuo	_____ / _____	

3.4 Lokalus Kubernetes klasteris ir GPU

Klasteris turi veikti kaip Docker konteineris (-iai) studento kompiuteryje ir būti aprašomas kaip kodas (konfigūracijos failas repozitorijoje ir/ar Terraform). Studentas renkasi laisvai; rekomenduojamos parinktys:

Įrankis	Konfigūracija kaip kodas	Terraform provider'is	GPU
`kind` (referencinis)	`kind-config.yaml`	`tehcyx/kind`	per NVIDIA `nvkind`: Docker numatytasis runtime → nvidia, Toolkit "volume-mounts" nustatymas (`nvidia-ctk config`), device plugin per Helm
`minikube` (docker driver'is)	`minikube start` flag'ai, addon'ai	`scott-the-programmer/minikube`	`--gpus all` (nuo v1.32) — paprasčiausias kelias; device plugin įjungiamas automatiškai
`k3d`	`k3d-config.yaml`	`pvotal-tech/k3d`	savas k3s vaizdas su CUDA; Pod'ams `runtimeClassName: nvidia`
Docker Desktop Kubernetes	nėra	nėra	nerekomenduojama (neaprašoma kaip kodas)

Rekomendacija: branduolys — CPU (`kind`); išplėstinė dalis — `minikube --gpus all` arba `nvkind`. Ištekliai: MLflow + PostgreSQL + inferencijos servisas ≈4 GB RAM; su Prometheus/Grafana ≈6 GB; silpnesniems kompiuteriams — GCP VM su tuo pačiu `kind`.

3.5 Užduočių kalendorius

Sav.	Data	Įvykis
1	09-02	Repozitorijų sukūrimas; U1 paskelbimas; modelio ir duomenų pasirinkimas
5	09-30	Lokalus klasteris laboratoriniame darbe (`kind` referencinis; `minikube --gpus all` demo)
6	10-07	Paskutinė U1 branduoliui reikalinga paskaita
7	10-14	U2 paskelbimas
8	10-21	**U1 pateikimas** ir gynimas per lab.; tarpinis atsiskaitymas; konsultacija (GPU/HPC, CI/CD)
9	10-28	Komandinio projekto temų suderinimas su lektoriumi
11	11-11	**U2 pateikimas** ir gynimas per lab.
12	11-18	Konsultacija: projekto statusas
15	12-09	Projekto pateikimas (tag `team-final`, ataskaita)
16	12-16	**Komandinio projekto gynimas** (Capstone Demo Day)

4 1 savaitė — praktika, tutorialai ir užduotys

Pilna praktinė medžiaga: seminaras (2 val.) + laboratorinis (3 val.) + savarankiškos užduotys.

4.0.1 Seminaras (2 val.) — detalus scenarijus

0:00–0:15 — Kurso administracija ir grupių formavimas.

- Pristatyti vertinimo sistemą, infrastruktūrą (GCP kreditai, HPC prieiga), GitHub naudojimo taisykles.
- Suformuoti grupes po 2–3 studentus; kiekviena grupė pasirenka **projekto pavadinimą** ir **komandinį GitHub org/repo**.

- Paaiškinti: kiekvienas laboratorinis darbas = vienas commit/tag semestro projekte.

0:15–0:45 — Atvejų analizė: MLOps nesėkmės. Case 1: Zillow Zestimate (2021).

- *Kontekstas:* Automatinis būstų kainų įvertinimas; iBuying programa.
- *Problema:* Modelis sistemingai **pervertino** namų kainas; algoritmas nebuvo pakankamai stebimas produkcijoje.
- *Pasekmės:* ∼$300M nuostoliai; 25 % darbuotojų atleista; programa sustabdyta.
- *Diskusijos klausimai:* Koks dreifo tipas? Kokie NFR nebuvo apibrėžti? Kokių stebėsenos įrankių trūko?

Case 2: Amazon rekomendacijų sistema.

- *Kontekstas:* 35 % pajamų priklauso nuo rekomendacijų.
- *Problema:* Sezoniškumas, vartotojų elgsenos pokyčiai; modelis nebuvo automatiškai perrenkamas.
- *Diskusijos klausimai:* Kaip apibrėžtumėte KPI? Koks būtų AIOps ciklas šiai sistemai?

Grupių užduotis (15 min.): Pasirinkite vieną atvejį ir užpildykite **RCA (Root Cause Analysis) šabloną**:

Laukas	**Ką rašyti**
Incidentas	Kas nutiko? Kada?
Techninė priežastis	Dreifas? Duomenys? Infrastruktūra?
Proceso spraga	Ko trūko MLOps procese?
MLOps sprendimas	Ką diegtumėte: monitoring, CI/CD, retraining?
NFR, kurie būtų apsaugoję	Latency? Drift tolerance? Alerting?

0:45–1:15 — Reikalavimų inžinerijos workshop. Dėstytojas pristato **gerą vs. blogą** reikalavimo pavyzdį:

Blogas reikalavimas	**Geras reikalavimas**
"Modelis turi būti tikslus"	"Accuracy $\geq$ 92 % test aplinkoje; F1 $\geq$ 0.88 klasėms A, B, C"
"Sistema turi būti greita"	"p95 inference latency $\leq$ 200 ms esant 100 RPS apkrovai"
"Sistema turi būti saugi"	"Visi PII laukai maskuojami; duomenys šifruojami AES-256; prieiga tik per RBAC"

Porų pratimas (20 min.): Kiekviena grupė parengia **3 funkcinius** ir **5 nefunkcinius** reikalavimus savo projektui. Reikalavimai turi atitikti SMART kriterijus (Specific, Measurable, Achievable, Relevant, Time-bound).

1:15–1:30 — Publikacijų sąrašas ir projekto pasirinkimas. Grupė pasirenka vieną publikaciją iš dėstytojo sąrašo. Pateikti:

- Publikacijos pavadinimas ir nuoroda.
- Problema, kurią spręsite analogiškai.
- Numatomas modelio tipas ir duomenų šaltinis.

4.0.2 Laboratorinis darbas (3 val.) — semestro projekto startas

Užduotis: Git repozitorija + reikalavimų dokumentas v1.0 + architektūros eskizas

Tikslas: Sukurti profesionalią projekto bazę, kurią semestro eigoje plėsite kiekvieną savaitę.

1 žingsnis: Repozitorijos struktūra (30 min.) Sukurkite GitHub repozitoriją su šia struktūra:

```
mlops-project/
  README.md
  .gitignore
  requirements.txt
  docs/
    requirements.md      # Gyvas reikalavimu dokumentas
    architecture.md      # Architekturos diagrama (Mermaid)
  src/
    .gitkeep             # Bus uzpildyta 2 sav.
  tests/
    .gitkeep
  infra/
    .gitkeep             # Bus uzpildyta 4 sav.
```

README.md turi turėti:

- Projekto pavadinimą ir 2–3 sakinių aprašymą.
- Komandos narius (vardai, GitHub username).
- Pasirinktos publikacijos nuorodą.
- Diegimo instrukcijas (placeholder).
- Licenciją (MIT).

.gitignore — Python ML projektui:

```
__pycache__/
*.pyc
.env
.venv/
data/raw/
data/processed/
models/
mlruns/
.dvc/cache/
*.egg-info/
.ipynb_checkpoints/
```

2 žingsnis: Reikalavimų dokumentas (60 min.) Užpildykite `docs/requirements.md` pagal šabloną:

```
# Projekto reikalavimai
Versija: 1.0
Data: YYYY-MM-DD
Autoriai: [vardai]
```

```
## 1. Projekto apimtis
### 1.1 Problemos aprasymas
[Kokia problema sprendziama? Verslo kontekstas?]

### 1.2 Stakeholderiai
| Stakeholderis | Interesas | Prioritetas |
|---------------|-----------|-------------|
| ...           | ...       | Aukstas     |

### 1.3 Use cases
| ID    | Use case          | Aktoriai  | Rezultatas       |
|-------|-------------------|-----------|------------------|
| UC-01 | ...               | ...       | ...              |

## 2. Funkciniai reikalavimai (FR)
| ID    | Reikalavimas                          | Prioritetas |
|-------|---------------------------------------|-------------|
| FR-01 | Sistema priima [formatas] ir grazina | Must        |
| FR-02 | ...                                   | Should      |

## 3. Nefunkciniai reikalavimai (NFR)
| ID     | Kategorija    | Reikalavimas              | Metrika/KPI        |
|--------|---------------|---------------------------|--------------------|
| NFR-01 | Nasumas       | p95 latency <= 200 ms     | Prometheus histogram|
| NFR-02 | Pralaidumas   | >= 100 RPS                | Load test          |
| NFR-03 | Dreifas       | Accuracy kritimas < 2%/30d| Evidently report   |
| NFR-04 | Saziningumas  | Demographic parity +/- 5% | Fairness test      |
| NFR-05 | Kastai        | Inference <= $0.001/req   | Cloud billing      |
| NFR-06 | Saugumas      | PII masking, TLS 1.3      | Security audit     |
| NFR-07 | Prieinamumas  | Uptime >= 99.9%           | Uptime monitor     |

## 4. KPI ir sekmes metrikos
### 4.1 Modelio KPI
- Accuracy >= ...
- F1 >= ...

### 4.2 Sistemos KPI
- p95 latency <= ...
- Error rate < 0.1%

### 4.3 Verslo KPI (jei taikoma)
- ...

## 5. Prielaidos ir apribojimai
- Duomenu saltinis: [aprasymas]
- Infrastruktura: GCP (be GPU) + HPC (GPU)
- Komandos dydis: 2-3 asmenys

## 6. Zinomos rizikos
| Rizika              | Tikimybe | Poveikis | Mitigacija        |
|---------------------|----------|----------|-------------------|
| Duomenu trukumas    | Vidutine | Aukstas  | Synthetic data    |

## 7. Keitimo istorija
| Versija | Data       | Autorius | Pakeitimai         |
|---------|------------|----------|--------------------|
| 1.0     | YYYY-MM-DD | ...      | Pradinis dokumentas|
```

Dažna klaida

Dažnos klaidos: per abstraktūs reikalavimai (“turi būti greita”), nėra KPI, nėra prioritetų (Must/Should/Could), neįtraukti saugumo ir fairness NFR.

3 žingsnis: Architektūros diagrama (45 min.) Sukurkite `docs/architecture.md` su Mermaid diagrama. Minimalus variantas:

```
# Projekto architektura (v1.0)

<!-- Mermaid diagrama - GitHub renderina automatikai -->
flowchart TB
    subgraph Data
        DS[(Duomenu saltinis)]
        DV[Duomenu validavimas]
    end
    subgraph Training
        EXP[MLflow eksperimentai]
        TRN[Modelio mokymas]
        REG[Model Registry]
    end
    subgraph Serving
        API[Inference API]
        MON[Stebesena]
    end
    DS --> DV --> TRN
    TRN --> EXP --> REG
    REG --> API --> MON
    MON -->|drift detected| TRN
```

Pastaba: `architecture.md` faile Mermaid bloką apgaubkite ```` ```mermaid ... ``` ```` sintakse. Diagrama turi apimti (net jei dar neimplementuota):

- Duomenų srautą (įėjimas → validavimas → mokymas).
- Modelio gyvavimo ciklą (eksperimentas → registras → serving).
- Stebėsenos ir feedback kilpą (drift → retraining).

4 žingsnis: Versijavimas ir pateikimas (15 min.)

```
git add .
git commit -m "feat: initial project setup and requirements v1.0"
git tag -a v1.0 -m "Week 1: requirements document and architecture"
git push origin main --tags
```

Pateikite dėstytojui:

- GitHub repo URL.
- Nuorodą į `docs/requirements.md` ir `docs/architecture.md`.

Užduoties rezultatas (deliverable)

- **D1:** GitHub repo su pilna struktūra (README, .gitignore, requirements.txt).
- **D2:** `docs/requirements.md` v1.0 — ≥3 FR, ≥5 NFR su KPI.
- **D3:** `docs/architecture.md` su Mermaid diagrama.
- **D4:** Git tag `v1.0`.

4.0.3 Vertinimo kriterijai (1 savaitė, iki 0,2 balo)

Kriterijus	Balai	Minimalus slenkstis
Repo struktūra ir README kokybė	0–0,03	README su komanda ir aprašymu
Funkciniai reikalavimai (≥3, su ID)	0–0,04	≥3 išmatuojami FR
Nefunkciniai reikalavimai (≥5, su KPI)	0–0,05	≥5 NFR su metrikomis
Architektūros diagrama	0–0,04	Mermaid su 3+ komponentais
Git tag v1.0 ir tvarkinga istorija	0–0,02	Tag egzistuoja
Atsiskaitymas (demo + klausimai + live edit)	0–0,02	Atsako ≥1 klausimą
Iš viso	**0–0,2**	**≥0,1 = atsiskaityta**

Atsiskaitymo klausimų pavyzdžiai:

1. Paaiškinkite vieną NFR ir kaip jį išmatuosite semestro eigoje.
2. Kuo skiriasi jūsų projekto duomenų dreifas nuo koncepcijų dreifo?
3. Parodykite `requirements.md` — pakeiskite vieną NFR pagal dėstytojo scenarijų (live edit).
4. Kodėl reikalavimai saugomi Git, o ne Google Docs?

4.0.4 Savarankiškas darbas (5 val.)

1. **Literatūra (2 val.):** Chip Huyen — *AI Engineering*, 1–3 skyriai. Užrašai: 5 pagrindinės mintys.
2. **Publikacija (1,5 val.):** Perskaityti pasirinktą publikaciją; parengti 1 puslapio santrauką `docs/paper-summary.md`.
3. **Reikalavimų tobulinimas (1 val.):** Grupės review: ar kiekvienas NFR turi KPI? Ar yra fairness ir security?
4. **Pasiruošimas 2 savaitei (0,5 val.):** Įdiegti MLflow lokaliai (`pip install mlflow`); peržiūrėti MLflow quickstart.

4.0.5 Tutorial 1.0 — Git repo sukūrimas nuo nulio

Komandų seka (vykdyti laboratorijoje)

```
# 1. GitHub: New repository -> mlops-fraud-detection (MIT license)

git clone git@github.com:<team>/mlops-fraud-detection.git
cd mlops-fraud-detection

mkdir -p docs src tests infra data/raw data/processed models
touch src/.gitkeep tests/.gitkeep infra/.gitkeep

# 2. .gitignore (kopijuoti is practice medziagos)

# 3. README.md - uzpildyti komanda, aprasymas, publikacija

git add .
git commit -m "chore: initial project scaffold"
git push origin main

# 4. Po requirements.md ir architecture.md:
git add docs/
git commit -m "feat: requirements v1.0 and architecture diagram"
git tag -a v1.0 -m "Week 1 deliverable"
git push origin main --tags
```

Tikėtinas rezultatas: GitHub repo su 2+ commit'ais, tag `v1.0`, `docs/` katalogas renderinasi.

4.0.6 Seminaro darbo lapas (1 savaitė)

Grupė / vardai	_____
Projekto pavadinimas	_____
Publikacija	_____
RCA: incidentas	
RCA: dreifo tipas	☐ Data ☐ Concept ☐ Code
Trūko NFR	
3 FR (trumpai)	1. _____ 2. _____ 3. _____
5 NFR + KPI	NFR-01 KPI: _____

4.0.7 Pilnas pavyzdys: užpildytas reikalavimų dokumentas

Tutorial 1.1 — fraud detection projekto `requirements.md` fragmentas

```
# Fraud Detection API - Reikalavimai v1.0
Data: 2026-09-15 | Autoriai: Ana, Benas, Greta

## 1.1 Problemos aprasymas
Sukurti realaus laiko fraud scoring API korteliu transakcijoms.
Tikslas: recall >= 0.92, false positive <= 0.5%.

## 2. Funkciniai reikalavimai
| ID    | Reikalavimas                              | Prioritetas |
|-------|-------------------------------------------|-------------|
| FR-01 | POST /score priima JSON transakcija       | Must        |
| FR-02 | Grazina fraud_score [0,1] ir decision     | Must        |
| FR-03 | decision=review siuncia i analitiko queue | Should      |

## 3. Nefunkciniai reikalavimai
| ID     | Kategorija | Reikalavimas           | KPI                      |
|--------|------------|------------------------|--------------------------|
| NFR-01 | Nasumas    | Inference latency      | p95 <= 80 ms @ 500 RPS |
| NFR-02 | Kokybe     | Sukciavimo aptikimas   | Recall >= 0.92         |
| NFR-03 | Kokybe     | False positive rate    | <= 0.5%                |
| NFR-04 | Dreifas    | Precision degradacija  | < 3% per 14 dienu      |
| NFR-05 | Saziningumas | TPR skirtumas regionu | <= 5%                   |
| NFR-06 | Kastai     | Inference kaina        | <= $0.0002/req        |
| NFR-07 | Prieinamumas | Uptime               | >= 99.95%              |
```

4.0.8 Papildomos praktinės užduotys (1 savaitė)

Praktinė užduotis

Užduotis 1.1 (lengva). Nukopijuokite fraud detection pavyzdį ir pritaikykite savo projektui: pakeiskite 3 NFR pagal savo problemą.
Užduotis 1.2 (vidutinė). Nubraižykite architektūros diagramą su **10 komponentų** (ne 6): pridėkite CI/CD, monitoring, Model Registry.
Užduotis 1.3 (sudėtinga). Parašykite `docs/rca-zillow.md`: pilną RCA ataskaitą Zillow atvejui (1 psl.).
Užduotis 1.4 (refleksija). Įvertinkite savo repo MLOps brandą (0–2). Ką pasieksite iki 6 savaitės?

5 2 savaitė — praktika, tutorialai ir užduotys

Pilna praktinė medžiaga. Teorija — `theory/02-reproducibility.tex`. Sesijos planas: 45 min. paskaita → pertrauka → pagrindinė dalis (planuotina trukmė ≈ 1 val. 45 min.), kurią visi baigia auditorijoje, → išplėstinis kelias (5 val. savarankiškai).

5.0.1 Apibendrinimas (10 min.) — atkartojamumo katastrofa

Įsivaizduokite scenarijų:

"Jūsų kolega išėjo atostogų. Jūs turite per 1 dieną pateikti jo geriausią modelį klientui. Jis paliko tik Jupyter notebook ir Slack žinutę 'naudok lr=0.001'. Ką darote?"

Užpildykite trūkstamų elementų sąrašą:

Elementas	Ar yra?	Kaip gauti su MLOps?
Tikslus kodas (commit)	?	Git
Duomenų versija	?	DVC hash
Hiperparametrai	?	MLflow params
Python/bibliotekų versijos	?	`log_model` aplinkos failai (`requirements.txt`)
Random seed	?	MLflow parametras (ne žyma — turi būti ieškomas)
Modelio failas	?	MLflow LoggedModel (`model_id`)

5.0.2 Laboratorinis darbas — MLflow + DVC pipeline

Užduotis: atkartojamas ML pipeline su MLflow tracking ir DVC versijavimu

Tikslas: bet kurį istorinį eksperimentą atkurti viena komanda `python reproduce.py <run_id>` per <**2 min.**, o rezultatas — statistiškai ekvivalentus (Δaccuracy $\leq$ 0,005).

Jei nedalyvavote 1 savaitės praktikoje

Šiai praktikai iš 1 savaitės tereikia dviejų dalykų: Git repozitorijos su projekto skeletu ir `docs/requirements.md` v1.0 (NFR lentelė). Pasivijimas — apie 10 min.:

```
# 1. Projekto skeletas (adresa nurodys destytojas)
git clone <kurso-repo> && cd fraud-detection

# 2. Aplinka - zr. 0 zingsni zemiau (venv + priklausomybes)

# 3. Duomenys ir pradinis modelis - vienas skriptas sugeneruoja viska:
python scripts/bootstrap.py
```

`bootstrap.py` sukuria `data/raw/dataset.csv` (sintetinės 1 000 transakcijų $\times$ 10 požymių, fiksuotas seed — visų studentų duomenys **identiški**), `data/processed/` splitą ir pradinį `models/model.pkl`. Splitas ir modelis yra tik užpildas — 2–3 žingsniuose juos perrašys jūsų pačių pipeline. `docs/requirements.md` v1.0 rasite repo skelete; jo turinį peržiūrėsite 6 žingsnyje.

0 žingsnis: aplinka ir priklausomybės (10 min.)

```
# Rekomenduojama (uv) - atsisiusti ir susipazinti:
#   https://docs.astral.sh/uv/
uv venv .venv && source .venv/bin/activate
uv pip install -r requirements.txt

# Fallback (paprastas pip):
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Patikra:
mlflow --version     # 3.15.2
dvc version          # 3.67.1
python scripts/bootstrap.py   # jei dar nepaleidote
```

`requirements.txt` (jau repo skelete):

```
mlflow==3.15.2
dvc==3.67.1
```

```
scikit-learn>=1.5
pandas>=2.0
pyyaml>=6.0
joblib>=1.3
psutil>=5.9           # sistemos metrikoms
```

Kodėl `==`, o ne `>=`? Nes aplinka yra viena iš penkių versijuojamų dimensijų: su "grindų" rėžiais kiekvienas studentas gautų vis kitą bibliotekų rinkinį.

1 žingsnis: MLflow Tracking Server (15 min.)

```
# Terminalas 1 - serveris (paliekame veikti):
mlflow server \
  --backend-store-uri sqlite:///mlflow.db \
  --artifacts-destination ./mlartifacts \
  --host 127.0.0.1 --port 5000

# Terminalas 2 - klientas:
export MLFLOW_TRACKING_URI=http://127.0.0.1:5000
```

Ką dabar gausite: metaduomenys (parametrai, metrikos, žymos) pateks į `mlflow.db` (SQLite), failai (modeliai, artefaktai) — į `mlartifacts/`. Katalogo `mlruns/` **nebus** — tai senesnės nei MLflow 3.x versijos standartas (senuose tutorial'uose jį matysite).

Pirmasis run'as — dar prieš DVC:

```
python src/train.py
```

Atsidarykite `http://127.0.0.1:5000` ir suraskite:

1. eksperimentą `fraud-detection` ir jame vieną run'ą (vardas `rf-n100-d5`);
2. **Parameters**: `n_estimators`, `max_depth`, `random_seed` — visi iš `params.yaml`;
3. **Tags**: `git_commit` ir `dvc_raw_md5` — antroji kol kas `untracked`, nes DVC dar neinicializuotas (pasikeis po 2 žingsnio!);
4. **Datasets**: `train` su duomenų santrauka (digest);
5. run'o skiltį **Models** → LoggedModel (`m-…`): aplinkos dimensija užfiksuota automatiškai — tarp modelio failų rasite `requirements.txt` ir `python_env.yaml`.

Dažna klaida

Jei 5000 portas užimtas (macOS AirPlay ir kt.): paleiskite su `--port 5001` ir atitinkamai pakeiskite `MLFLOW_TRACKING_URI`. Žr. troubleshooting lentelę.

2 žingsnis: DVC — duomenų versijavimas (20 min.) Atkreipkite dėmesį: šios savaitės failus kopijuojatės į **savo** Git repozitoriją (1 savaitės repo arba naują) — kurso repozitorija yra tik šaltinis, ne jūsų darbo vieta, ir pakeitimai joje negalimi. Be savo asmeninės Git repozitorijos tolesnių DVC žingsnių atlikti negalėsite (`dvc init` reikalauja Git repo šaknies). Jei savo repozitorijos dar neturite:

```
# 1. GitHub'e sukurkite tuscia PRIVACIA repozitorija (be README)
# 2. Lokaliai susikurkite darbo kataloga is kurso failu:
mkdir fraud-detection && cd fraud-detection
cp -r <kurso-repo>/tutorial/fraud-detection/. .
rm -rf .git                        # kopija neturi likti svetimu klonu
git init -b main
```

```
git add . && git commit -m "week 2 skeleton"
# 3. Susiekite su GitHub (adresa rasite zaliame mygtuke "Code"):
git remote add origin git@github.com:<vartotojas>/<repo>.git
git push -u origin main
```

Toliau — DVC su paprasčiausiu remote (katalogu):

```
dvc init                # sukuria .dvc/ ir auto-stage'ina jo failus
mkdir -p ~/dvcstore     # paprasciausias remote - katalogas
dvc remote add -d localremote ~/dvcstore
git add .dvc/config     # remote konfiguracija keliauja i Git!
git commit -m "dvc: init + local remote"

dvc add data/raw/dataset.csv   # baitai -> cache, pointer'is -> Git
git add data/raw/dataset.csv.dvc
git commit -m "data: v1"

dvc push                        # baitai -> remote
git push                        # pointer'iai -> jusu Git serveris
```

Ką ką tik padarėte — **du push'ai** (paskaitos 2.4 skyrius veiksme): Git perkėlė apie 100 baitų pointer'į, DVC — tikruosius duomenis. Pasitikrinkite:

1. `cat data/raw/dataset.csv.dvc` — pamatysite `md5: …` santrauką (visų grupės studentų ji **identiška** — kodėl?);
2. `ls ~/dvcstore/files/md5/` — baitai remote'e guli ne vardu, o santraukos adresu;
3. `dvc status` ir `dvc status -c` — abu švarūs.

Tikri remote'ai skiriasi tik `dvc remote add` eilute ir autentifikacija: SSH/MinIO/S3/GCS atveju `mkdir` žingsnio nėra (saugykla jau egzistuoja), kiekvienam backend'ui reikia papildomo paketo (`dvc[ssh]`, `dvc[s3]`, `dvc[gs]`), o paslaptys rašomos per `dvc remote modify --local` — į Git jos nepatenka. Pilnos komandos — Tutorial 2.7 (išplėstinis kelias); visa kita (`dvc push` / `dvc pull`) veikia identiškai.

3 žingsnis: konfigūracija ir pipeline (15 min.) `params.yaml` — be fiktyvaus `learning_rate` (RandomForest jo neturi!):

```
train:
  n_estimators: 100
  max_depth: 5
  random_seed: 42
```

`dvc.yaml` — atkreipkite dėmesį į dvitaškį ir `cache: false` eilutes:

```
stages:
  prepare:
    cmd: python src/prepare_data.py
    deps:
      - src/prepare_data.py
      - data/raw/dataset.csv
    outs:
      - data/processed/train.csv
      - data/processed/test.csv
  train:
    cmd: python src/train.py
    deps:
      - src/train.py
```

```
      - data/processed/train.csv
      - data/processed/test.csv
    params:
      - params.yaml:        # demesio: dvitaskis!
    outs:
      - models/model.pkl
      - models/run_id.txt:
          cache: false      # failas lieka Git'e (zr. akcentus zemiau)
  evaluate:
    cmd: python src/evaluate.py
    deps:
      - src/evaluate.py
      - models/model.pkl
      - models/run_id.txt
      - data/processed/test.csv
    metrics:
      - metrics.json:
          cache: false      # skaito dvc metrics diff ir 6 sav. vartai
```

Trys akcentai: (1) `params:` sąrašo elementas su **dvitaškiu** — dažniausia sintaksės klaida; (2) `cache: false` — maži tekstiniai failai paliekami Git'e, kad juos matytų žmonės ir CI; (3) `models/run_id.txt` — lineage jungtis: pipeline'as pats užsirašo, kurį MLflow run'ą sukūrė. Šio pasirinkimo kaina: stadija su `cache: false` išvestimi nepatenka į DVC run-cache, tad grąžinę seną parametro reikšmę mokymą kartosite iš naujo (kelios sekundės), o ne atkursite akimirksniu — sąmoningas mainas: matomumas Git'e vietoj spartos bonuso.

4 žingsnis: mokymo skriptas su MLflow (25 min.) `src/prepare_data.py` — skaido duomenis ir išsaugo train/test CSV (nekeičiamas; pilnas kodas — Tutorial 2.1).

`src/train.py` — esminė logika (pilnas failas repo skelete):

```
import argparse, os, subprocess
from pathlib import Path
import joblib, mlflow, mlflow.sklearn, pandas as pd, yaml
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

ROOT = Path(__file__).resolve().parents[1]

def git_commit() -> str:
    try:
        cmd = ["git", "rev-parse", "--short=8", "HEAD"]
        out = subprocess.check_output(cmd, cwd=ROOT, text=True)
        return out.strip()
    except Exception:
        return "unknown"

def dvc_md5(dvc_file: Path) -> str:
    """Duomenu md5 is pointer'io - jungtis i dvc.lock."""
    try:
        return yaml.safe_load(dvc_file.read_text())["outs"][0]["md5"]
    except Exception:
        return "untracked"

def main() -> None:
    cfg = yaml.safe_load((ROOT / "params.yaml").read_text())
    params = cfg["train"]
    parser = argparse.ArgumentParser()
    parser.add_argument("--n_estimators", type=int)
```

```
    parser.add_argument("--max_depth", type=int)
    parser.add_argument("--random_seed", type=int)
    for key, value in vars(parser.parse_args()).items():
        if value is not None:
            params[key] = value   # CLI perraso params.yaml reiksme

    mlflow.set_experiment("fraud-detection")
    train = pd.read_csv(ROOT / "data/processed/train.csv")
    test = pd.read_csv(ROOT / "data/processed/test.csv")
    x_train, y_train = train.drop("target", axis=1), train["target"]
    x_test, y_test = test.drop("target", axis=1), test["target"]

    run_name = f"rf-n{params['n_estimators']}-d{params['max_depth']}"
    with mlflow.start_run(run_name=run_name) as run:
        mlflow.log_params(params)   # konfiguracija, iskaitant seed
        raw_md5 = dvc_md5(ROOT / "data/raw/dataset.csv.dvc")
        # zymos - jungties raktai i Git ir DVC:
        mlflow.set_tags({"git_commit": git_commit(),
                         "dvc_raw_md5": raw_md5,
                         "model_family": "RandomForestClassifier"})
        # duomenu kilme (vardas + digest):
        mlflow.log_input(mlflow.data.from_pandas(
            train, source="data/processed/train.csv",
            targets="target", name="train"), context="training")

        model = RandomForestClassifier(
            n_estimators=params["n_estimators"],
            max_depth=params["max_depth"],
            random_state=params["random_seed"])
        model.fit(x_train, y_train)
        preds = model.predict(x_test)
        acc = accuracy_score(y_test, preds)
        f1 = f1_score(y_test, preds, average="weighted")

        # modelis + aplinkos failai (LoggedModel):
        info = mlflow.sklearn.log_model(model, name="model",
                                        input_example=x_train.head(3))
        # metrika <-> modelis:
        mlflow.log_metric("accuracy", acc, model_id=info.model_id)
        mlflow.log_metric("f1", f1, model_id=info.model_id)

        os.makedirs(ROOT / "models", exist_ok=True)
        # DVC sekamas artefaktas velesnems savaitems:
        joblib.dump(model, ROOT / "models/model.pkl")
        run_id_file = ROOT / "models/run_id.txt"
        run_id_file.write_text(run.info.run_id + "\n")
        print(f"run_id={run.info.run_id} model_id={info.model_id}"
              f" accuracy={acc:.4f} f1={f1:.4f}")
```

Kas čia kitaip nei senuose pavyzdžiuose internete (paskaitos 2.3 sąrašas praktikoje): `log_model` su `name=` ir `input_example` (LoggedModel + signatūra + aplinka), `model_id=` metrikose, `log_input` digest'as, seed kaip **parametras**, žymos — jungties raktai į Git ir DVC.

`src/evaluate.py` — skaito modelį, rašo `metrics.json` **ir prisega vertinimo metrikas prie mokymo run'o**:

```
import json
from pathlib import Path
import joblib, mlflow, pandas as pd
```

```
from sklearn.metrics import (accuracy_score, f1_score,
                             precision_score, recall_score)

ROOT = Path(__file__).resolve().parents[1]

def main() -> None:
    model = joblib.load(ROOT / "models/model.pkl")
    test = pd.read_csv(ROOT / "data/processed/test.csv")
    x, y = test.drop("target", axis=1), test["target"]
    preds = model.predict(x)
    metrics = {"accuracy": accuracy_score(y, preds),
               "f1": f1_score(y, preds, average="weighted"),
               "precision": precision_score(y, preds),
               "recall": recall_score(y, preds)}
    out = json.dumps(metrics, indent=2) + "\n"
    (ROOT / "metrics.json").write_text(out)   # skaito 6 sav. vartai
    print(out)

    run_id_file = ROOT / "models/run_id.txt"
    if run_id_file.exists():   # vertinimas gula ant mokymo run'o
        try:
            run_id = run_id_file.read_text().strip()
            client = mlflow.MlflowClient()
            for key, value in metrics.items():
                client.log_metric(run_id, f"eval/{key}", value)
        except Exception as exc:   # serveris gali buti nepasiekiamas
            print(f"warning: eval metrics not attached ({exc})")
```

Paleiskite pilną pipeline'ą ir commit'inkite jo įrašus:

```
dvc repro     # prepare -> train -> evaluate (visos trys stadijos)
git add dvc.lock metrics.json models/run_id.txt
git commit -m "experiment 1: baseline"
```

Pasitikrinkite: `dvc repro` dar kartą — visos stadijos turi būti praleistos ("didn't change, skipping"); MLflow UI — run'as dabar turi ir `eval/*` metrikas; `cat dvc.lock` — kiekvienai stadijai užfiksuoti visų įvesčių md5 ir parametrai.

5 žingsnis: eksperimentai ir reprodukcija (35 min.) Atlikite bent tris eksperimentus — keiskite `params.yaml` (arba CLI vėliavėlę `--n_estimators`) ir kartokite ciklą:

```
# 2 eksperimentas:
#   params.yaml: n_estimators: 100 -> 300
dvc repro    # persileidzia tik train + evaluate. Kodel ne prepare?
git add params.yaml dvc.lock metrics.json models/run_id.txt
git commit -m "experiment 2: n_estimators=300"

# 3 eksperimentas:
#   params.yaml: max_depth: 5 -> 2 (arba jusu pasirinkimas)
dvc repro
git add params.yaml dvc.lock metrics.json models/run_id.txt
git commit -m "experiment 3: max_depth=2"

dvc metrics diff HEAD~1            # metriku pokytis tarp commit'u
dvc params diff HEAD~1
dvc push                           # nepamirskite antrojo push'o!
```

MLflow UI: pažymėkite visus tris run'us → **Compare**. Kuris parametras šiam duomenų rinkiniui iš tiesų keičia metrikas, o kuris — beveik ne? (Pabandykite sau atsakyti.)

Reprodukcija — paskaitos 2.6 SLA veiksme. `reproduce.py` (pilnas failas repo skelete) logika — penki žingsniai:

```
run = mlflow.get_run(run_id)          # 0. irasas (zymos, parametrai)
git checkout <run.tags.git_commit>    # 1. to momento kodas
dvc checkout --force <*.dvc failai>   # 2. tie patys duomenu baitai
# 3. params.yaml perrasomas run'o parametrais
dvc repro --force                     # 4. pilnas DAG -> naujas run
# 5. accuracy/f1 palyginimas su tolerancija (zyma reproduces=<id>);
#    git checkout atgal - repo lieka kaip radote
```

```
time python reproduce.py <pirmojo-eksperimento-run-id>
```

Ko tikėtis: naujas run'as UI su žyma `reproduces=<senasis id>`, abu delta lygūs nuliui (CPU + sklearn + tas pats seed = bitų tikslumu identiška), laikas — toli iki 2 min. ribos.

6 žingsnis: reikalavimų atnaujinimas v1.1 (10 min.) Prieš pildydami — greita porų peržiūra: parodykite kolegai vieną savo v1.0 NFR; ar jis turi KPI? Ar matuojamas? Tada pridėkite į `docs/requirements.md`:

```
## 3.x Eksperimentu sekimo ir atkartojamumo NFR (v1.1)

| ID     | Reikalavimas                              | KPI          |
|--------|-------------------------------------------|--------------|
| NFR-08 | Istorinis run atkuriamas reproduce.py     | 2 min, 0.005 |
| NFR-09 | Run: git commit + duomenu md5 zymos       | 100 % run'u  |
| NFR-10 | dvc.lock md5 = run'o dvc_raw_md5          | 100 % run'u  |

## 7. Keitimo istorija
| 1.1 | <data> | <autoriai> | MLflow + DVC pipeline, NFR-08..10 |
```

```
git add docs/requirements.md
git commit -m "docs: requirements v1.1 (tracking NFR)"
git tag -a v1.1 -m "Week 2: MLflow + DVC pipeline"
git push origin main --tags    # jei turite remote
```

Užduoties rezultatas (deliverable)

- **D1:** veikiantis `dvc.yaml` pipeline (prepare → train → evaluate); pakartotinis `dvc repro` nieko neperskaičiuoja.
- **D2:** ≥3 MLflow run'ai su skirtingais **realiais** hiperparametrais, matomi UI, kiekvienas su `git_commit` ir `dvc_raw_md5` žymomis.
- **D3:** `python reproduce.py <run_id>` atkuria pirmą eksperimentą per <2 min., Δaccuracy $\leq$ 0,005.
- **D4:** `docs/requirements.md` v1.1 + Git tag `v1.1`.
- **D5:** DVC remote sinchronizuotas (`dvc push` atliktas; **bet kuris** backend'as — katalogas, SSH, MinIO, GCS).

5.0.3 Savęs vertinimo kriterijai — ką turėtumėte turėti po 2 savaitės

Šios savaitės laboratorinis darbas atskirai nevertinamas — lentelė žemiau yra jūsų pačių kontrolinis sąrašas.

Kriterijus	Kaip įsitikinti
DVC pipeline su 3 stage'ais	`dvc repro` veikia; `dvc.lock` commit'intas
MLflow logging (params, metrics, tags, dataset, model)	≥3 run'ai UI su jungties žymomis
`python reproduce.py` veikia	<2 min., Δaccuracy ≤ 0,005
DVC remote sinchronizuotas (bet kuris backend'as)	`dvc push` atliktas
`requirements.md` v1.1 su tracking NFR	NFR-08–NFR-10 su KPI

Savęs pasitikrinimo klausimai:

1. Gebėti atsidaryti MLflow UI — paaiškinti, kuris run'as geriausias ir kodėl. Kuo skiriasi jo `accuracy` ir `eval/accuracy`?
2. Ką saugo `data/raw/dataset.csv.dvc`, o ką — remote saugykla? Kur yra tikrieji baitai?
3. Kas pasikeistų `dvc.lock`, jei pakeistumėte `params.yaml`? Kurios stadijos persileistų pridėjus 100 eilučių prie žaliavos — ir kodėl?
4. Paleiskite `python reproduce.py <run_id>` — paaiškinkite sau kiekvieną iš penkių žingsnių.
5. Pasižiūrėkite run'o git commit'ą, duomenų santrauką ir aplinkos failus — kur kiekvienas gyvena?
6. Kodėl yra `mlflow.db`, bet nėra `mlruns/` katalogo?

5.0.4 Savarankiškas darbas (5 val.) — išplėstinis kelias

1. **Hydra (1,5 val.):** perkelkite `params.yaml` į `conf/` medį (paskaitos 2.5 pavyzdys); paleiskite `python train.py --multirun max_depth=2,5,10`; sujunkite su MLflow (`log_params` + `log_dict`). Palyginkite: kuo `--multirun` + tracker'is pranašesnis už tris rankinius paleidimus?
2. **W&B su LLM fine-tuning (2 val.):** Tutorial 2.6 — Qwen3-0.6B distiliacija su gyvu W&B monitoringu. Tas pačias sąvokas (run, params, metrikos su step, sistemos paneliai) atpažinsite kitame įrankyje — paskaitos 2.5 palyginimo lentelė veiksme.
3. **Nuotolinis DVC remote (1 val.):** pakeiskite katalogo remote į SSH arba MinIO/S3 pagal Tutorial 2.7; `dvc push` — semantika nesikeičia.
4. **Docker MLflow serveris (0,5 val., paruošia 3 savaitei):** `ghcr.io/mlflow/mlflow` konteineris su tomu; U1 užduoties pirmasis laiptelis.
5. **`dvc exp` (papildomai):** `dvc exp run -S train.max_depth=8 --temp`, `dvc exp show` — eksperimentai be commit'ų; palyginkite su MLflow run'ais.

5.0.5 Tutorial 2.1 — pilnas `prepare_data.py`

Duomenų paruošimo script'as

```python
"""Prepare train/test split. Run: python src/prepare_data.py"""
import os
import pandas as pd
from sklearn.model_selection import train_test_split

RAW = "data/raw/dataset.csv"
OUT_DIR = "data/processed"
TEST_SIZE = 0.2
SEED = 42

def main():
    os.makedirs(OUT_DIR, exist_ok=True)
    df = pd.read_csv(RAW)
    train, test = train_test_split(
        df, test_size=TEST_SIZE, random_state=SEED, stratify=df["target"]
    )
    train.to_csv(f"{OUT_DIR}/train.csv", index=False)
    test.to_csv(f"{OUT_DIR}/test.csv", index=False)
    print(f"Train: {len(train)}, Test: {len(test)}")

if __name__ == "__main__":
    main()
```

5.0.6 Tutorial 2.2 — žingsnis po žingsnio pipeline paleidimas

1. **Terminalas 1** — serveris: `mlflow server --backend-store-uri sqlite:///mlflow.db --artifacts-destination ./mlartifacts --port 5000`
2. **Terminalas 2** — `export MLFLOW_TRACKING_URI=http://127.0.0.1:5000`
3. `dvc repro` — turi įvykti trys dalykai: (a) trys stadijos paleidžiamos iš eilės su savo komandomis; (b) `train` stadija atspausdina `run_id=… model_id=… accuracy=… f1=…` eilutę ir nuorodą į run'ą; (c) atsiranda arba atsinaujina `dvc.lock`.
4. Pakartokite `dvc repro` nieko nekeitę — visos trys stadijos turi būti **praleistos**. Jei kuri nors persileidžia, kažkas nefiksuota — išsiaiškinkite kas (dažniausia priežastis: išvestis pakeista ranka).
5. `dvc dag` — pipeline grafas; `cat dvc.lock` — visų įvesčių/išvesčių santraukos; UI — run'as su `eval/*` metrikomis.

5.0.7 Tutorial 2.3 — `.dvc` pointer failo anatomija

```
outs:
- md5: <32 hex simboliai>
  size: <baitai>
  hash: md5
  path: dataset.csv
```

Git saugo tik šį apie 100 baitų failą. Tikrieji baitai — cache ir remote, adresuojami santraukos: `~/dvcstore/files/md5/<pirmi 2 simboliai>/<likusieji>`. `dvc push` / `dvc pull`

sinchronizuoja; parsiųsti baitai per-suskaičiuojami — sugadintas failas neprasmuks (paskaitos "matematinė garantija").

5.0.8 Tutorial 2.4 — prisijungimas prie dėstytojo serverio ("at scale" akimirka)

Pagrindinės dalies įterpinys po 5 žingsnio (10 min., jei klasės serveris pasiekiamas): tas pats kodas, kitas adresas.

```
export MLFLOW_TRACKING_URI=http://<destytojo-adresas>:5000
export MLFLOW_TRACKING_USERNAME=studentXX      # is dalijamo lapo
export MLFLOW_TRACKING_PASSWORD=<slaptazodis>
python src/train.py
```

Ką pamatysite: (1) jūsų run'as atsiranda **bendrame** eksperimente tarp visos grupės run'ų — stulpelis `mlflow.user` sako, kieno kuris; (2) dėstytojo LLM demo run'e — gyvos `system/gpu_0_*` kreivės; (3) kodas **nepasikeitė nė viena eilute** — tik du aplinkos kintamieji. Tai ir yra paskaitos "at scale" tezė. Baigę grąžinkite `MLFLOW_TRACKING_URI` į lokalų serverį.

5.0.9 Tutorial 2.5 — savo duomenys: Hugging Face → parquet → DVC

Semestro projekte kurso fraud pavyzdžio naudoti negalima — duomenis rinksitės patys (atviri HF rinkiniai rekomenduojami). Kad jūsų pipeline'as liktų atkartojamas:

```
from datasets import load_dataset
# 1. fiksuokite revizija!
ds = load_dataset("<jusu-rinkinys>", revision="<commit-arba-tag>")
ds["train"].to_parquet("data/raw/train.parquet")  # 2. eksportas
```

```
dvc add data/raw/train.parquet   # 3. toliau - kaip su dataset.csv
git add data/raw/train.parquet.dvc
git commit -m "data: <rinkinys> @ <revizija>"
dvc push
```

Kodėl neužtenka vien `revision=`? HF fiksuoja *šaltinį*, bet ne jūsų *apdorotą* variantą (filtravimą, split'us). DVC santrauka fiksuoja būtent tuos baitus, kuriuos matė `train.py`. Abi nuorodos kartu — pilna duomenų kilmė.

5.0.10 Tutorial 2.6 (išplėstinis kelias) — LLM fine-tuning su W&B monitoringu

Tas pats eksperimentų sekimo pasaulis, bet: tikras LLM (Qwen3-0.6B), tikra GPU apkrova ir kitas tracker'is — Weights & Biases. Receptas — S. Raschka *Build a Reasoning Model (From Scratch)* 8 skyrius (distiliacija: mokome bazinį modelį iš DeepSeek-R1 samprotavimo pėdsakų); knyga metrikas rašo į CSV, mes tą patį ciklą instrumentuojame `wandb.log`.

Reikalavimai: NVIDIA GPU su apie 10 GB laisvos VRAM (su `--max_seq_len 512` telpa ir į mažesnes); be GPU — žr. pastabą apačioje.

1. Aplinka (atskiras venv — kitos priklausomybės nei fraud projekto):

```
cd wb_monitoring   # finetune_qwen.py katalogas (nurodys destytojas)
uv venv .venv && source .venv/bin/activate
# fiksuotos versijos: torch, wandb, reasoning-from-scratch
uv pip install -r requirements.txt
```

2. W&B paskyra — vienas iš trijų kelių:

- turite paskyrą: `wandb login` ir įklijuokite raktą iš `https://wandb.ai/authorize`;
- neturite: nemokama registracija užtrunka apie 2 min. (akademinis planas — nemokamas);

- nenorite paskyros: `export WANDB_MODE=offline` — viskas veikia, įrašai guls lokaliai, vėliau `wandb sync wandb/offline-run-*`.

3. Dūmų testas (1–2 min., išbando visą grandinę; pirmas paleidimas parsisiųs duomenų rinkinį apie 107 MB ir modelį apie 1,5 GB — abu cache'inami kataloge):

```
python finetune_qwen.py --train_size 12 --val_size 4 --log_every 4
```

Terminale turi pasirodyti W&B run'o nuoroda; atsidarę ją matysite pirmas `train/loss` reikšmes.

4. Pilnas lab run'as (300 žingsnių; trukmė priklauso nuo jūsų GPU):

```
python finetune_qwen.py                # cosine lr, max_seq_len 1024
# variantai:
python finetune_qwen.py --schedule constant   # knygos pastovus lr
python finetune_qwen.py --watch_gradients   # + gradientu histogramos
python finetune_qwen.py --max_seq_len 512     # mazesnei GPU atminciai
```

5. Ką stebėti UI: `train/loss` (triukšminga pagal konstrukciją — kodėl?), `val/loss` (tikrasis signalas — turi kristi), `train/lr` (warm-up + cosine forma), `train/grad_norm` (šuoliai = kam egzistuoja `--grad_clip`), `perf/tokens_per_s`, **System** panelis (GPU utilizacija ir atmintis — W&B renka automatiškai), `samples/before_after` lentelė (ką apie 300 distiliacijos žingsnių realiai pakeičia modelio atsakymuose).

6. Sąvokų tiltas — užpildykite patys: kas W&B atitinka MLflow `experiment` / `run` / `log_params` / `log_metric(step=)` / sistemos metrikas? (Paskaitos 2.5 lentelė.)

Pastabos: be GPU šio tutorial'o daryti neverta (CPU treniruotė truktų valandas) — W&B sąvokas galite išbandyti savarankiško darbo 1–2 užduotyse su fraud modeliu (`wandb.init` + `wandb.log`); jei `CUDA out of memory` — mažinkite `--max_seq_len`; jei tinklas blokuoja W&B — offline režimas.

5.0.11 Tutorial 2.7 (išplėstinis kelias) — tikri DVC remote'ai: SSH, MinIO/S3, GCS

Tikram remote'ui `mkdir` žingsnio nėra — saugykla jau egzistuoja (serverio katalogas, bucket'as). Keičiasi tik `dvc remote add` eilutė ir autentifikacija.

SSH (katalogas nuotoliniame serveryje):

```
dvc remote add -d myremote ssh://user@serveris.lt/home/user/dvcstore
uv pip install 'dvc[ssh]'          # papildoma priklausomybe
# autentifikacija: SSH raktai, arba:
dvc remote modify --local myremote keyfile ~/.ssh/id_ed25519
```

MinIO / S3:

```
dvc remote add -d myremote s3://mano-bucket/dvcstore
# endpointurl reikia tik MinIO; tikram AWS S3 - nereikia:
dvc remote modify myremote endpointurl http://minio.serveris.lt:9000
uv pip install 'dvc[s3]'
# kredencialai NE i Git: --local raso i .dvc/config.local (gitignored)
dvc remote modify --local myremote access_key_id AKIA...
dvc remote modify --local myremote secret_access_key ...
```

GCS:

```
dvc remote add -d myremote gs://mano-bucket/dvcstore
uv pip install 'dvc[gs]'
gcloud auth application-default login   # kredencialai per gcloud
```

Visa kita identiška: `git add .dvc/config`, commit, `dvc push` / `dvc pull` veikia taip pat. Du akcentai: (1) kiekvienam backend'ui — papildomas paketas (`dvc[ssh]`, `dvc[s3]`, `dvc[gs]`; bazinis `dvc` iš `requirements.txt` dengia tik lokalų/HTTP); (2) skirtis tarp `dvc remote modify` (bendra konfigūracija, commit'inama) ir `dvc remote modify --local` (paslaptys, į Git nepatenka) — ta pati "jokių paslapčių repozitorijoje" taisyklė kaip visame kurse.

5.0.12 Troubleshooting — dažnos problemos (2 savaitė)

Klaida	Priežastis	Sprendimas
`Connection refused :5000`	MLflow serveris neveikia	Paleisti `mlflow server` 1-ame terminale
`[Errno 98] address already in use` arba keista 404	5000 portas užimtas (macOS AirPlay ir kt.)	`--port 5001` + atitinkamas `MLFLOW_TRACKING_URI`
`403 Invalid Host header`	Serveris pasiektas ne per localhost vardą	`--allowed-hosts` (arba junkitės per `127.0.0.1`)
`Parameters 'params.yaml' are missing`	`dvc.yaml` be dvitaškio po `params.yaml`	`- params.yaml:` (su dvitaškiu!)
`output ... is already tracked by SCM`	failas jau Git'e	`git rm -r --cached <failas>` ir kartokite `dvc add`
Po nepavykusio `dvc repro` dingo `models/model.pkl`	DVC ištrina stadijos išvestis prieš ją leisdamas	sutvarkykite klaidą ir paleiskite `dvc repro` iš naujo
`artifact_path is deprecated` įspėjimas	senas `log_model` kvietimas	nekritinis; naudokite `log_model(model, name=…)`
Nėra `mlruns/` katalogo	MLflow 3.x su serveriu jo nekuria	metaduomenys — `mlflow.db`, failai — `mlartifacts/`
Skirtingi accuracy tarp kolegų	nefiksuotas seed arba kiti duomenys	`random_seed` parametre + `dvc_raw_md5` žymų palyginimas
`reproduce.py` neranda run'o	neteisingas ID arba kitas serveris	kopijuokite ID iš UI; pasižiūrėkite `MLFLOW_TRACKING_URI`

5.0.13 Papildomos praktinės užduotys (2 savaitė)

Praktinė užduotis

Užduotis 2.1 (lengva). Po 3 run'us keisdami `n_estimators` (50/100/300) ir `max_depth` (2/5/10). MLflow UI Compare: kuris parametras šiems duomenims turi realią įtaką, o kuris — ne? Suformuluokite vieno sakinio išvadą.

Užduotis 2.2 (vidutinė). Pakeiskite `data/raw/dataset.csv` (pridėkite 100 eilučių). Paleiskite `dvc repro`. Kokie stage'ai persiskaičiavo? Kodėl? Ką dabar rodo `dvc status -c` ir ko reikia, kad kolega gautų naują versiją?

Užduotis 2.3 (sudėtinga). Parašykite `docs/experiment-log.md` lentelę: run ID, params, accuracy, f1, git commit — visi 3+ eksperimentai.

Užduotis 2.4 (refaktorinimas). Atsidarykite savo run'o LoggedModel failus (`MLmodel`, `requirements.txt`, `python_env.yaml`). Įdiekite modelį iš registro: `mlflow.sklearn.load_model("models:/<vardas>/<versija>")` — prieš tai užregistruokite run'o modelį UI mygtuku "Register model".

Užduotis 2.5 (bonus). Įdiekite Hydra; paleiskite `--multirun` su 3 `max_depth` reikšmėmis.

6 3 savaitė — Konteinerizacija: praktika

Tikslas

Sukurti CPU ir ONNX vaizdus, patikrinti API, dydį bei saugumą ir publikuoti GHCR. Pagrindinei sekai pakanka studentų repo esančio CPU/ONNX rinkinio. GPU ir CI užduotims reikia atskirai pateikiamų failų bei aplinkos.

6.0.1 0. Pasiruošimas prieš užsiėmimą

Reikia Docker su Buildx, Python 3.10+, Git, curl, Trivy ir GitHub CLI (`gh`). Trivy tikrina vaizdų pažeidžiamumus, o `gh` naudojamas prisijungti prie GHCR. Jei jų dar neturite, pirmiausia atlikite **šio PDF Ubuntu paruošimo priedą**, p. 35. Mac kompiuteryje reikia paleisto Docker Desktop ir tų pačių CLI įrankių; Ubuntu diegimo komandų Mac nevykdykite.

Naudokite **savo Git repozitoriją su atnaujintais** `tutorial/fraud-detection` **failais**. Vien senos 2 savaitės kopijos nepakanka. Prieš perimdami kurso atnaujinimus išsaugokite savo pakeitimus. GitHub paskyra turi turėti teisę siųsti pakeitimus į jūsų repo; bendras kurso repo skirtas medžiagai pasiimti.

Savo repo terminale pradėkite Bash ir pereikite į repo šaknį:

```
bash
cd "$(git rev-parse --show-toplevel)"
```

Tik jei kopijavote visą kursą ir jame yra šis pakatalogis, vykdykite:

```
cd tutorial/fraud-detection
```

Jei į savo repo perkėlėte tik `fraud-detection` turinį ir jo šaknyje jau yra `docker/`, `src/`, `scripts/`, paskutinę `cd` komandą praleiskite. Toliau dirbkite tame pačiame terminale. Pradinė patikra:

```
docker info
python3 --version
trivy --version
gh --version
```

Visos keturios komandos turi pavykti. Kopijuokite komandas iš kodo blokų; eilutę su \ tęskite kitoje eilutėje. **Ties klaida sustokite.** Viengubas - ir dvigubas -- yra skirtingi parametrų žymekliai; jų nekeiskite ilgu tipografiniu brūkšniu.

Dažna klaida

Paruošimas perrašo demonstracinius `models/` modelius ir sintetinius CSV. Ankstesnius savo duomenis išsaugokite kitur. Pirmam build reikia interneto, bent 8 GB RAM ir 15 GB disko CPU/ONNX daliai.

6.0.2 1. CPU ir ONNX: build, API ir saugumas

Failai jau paruošti repo. Peržiūrėkite `docker/Dockerfile.single`, `docker/Dockerfile.cpu`, `docker/Dockerfile.onnx` ir projekto šaknies `.dockerignore`. Toliau pateiktas komandas kopijuokite iš eilės; atskiro jas paleidžiančio shell skripto nereikia.

```
#!/usr/bin/env bash
# Run from tutorial/fraud-detection. Every command is also safe to paste separately.
set -eu
mkdir -p models data
```

```
# Use the same Python and sklearn versions for training and container serving.
docker build -f docker/Dockerfile.prepare -t mlops-prepare:week3 .
docker run --rm --user "$(id -u):$(id -g)" \
  -v "$PWD/models:/app/models" -v "$PWD/data:/app/data" mlops-prepare:week3

docker build -f docker/Dockerfile.single -t mlops-single:local .
docker build -f docker/Dockerfile.cpu -t mlops-cpu:local .
docker build -f docker/Dockerfile.onnx -t mlops-onnx:local .

# These commands return once HTTP and Docker HEALTHCHECK are ready.
python3 scripts/week3.py start cpu --port 18080
python3 scripts/week3.py start onnx --port 18081
curl -fsS http://127.0.0.1:18080/health
curl -fsS http://127.0.0.1:18080/predict \
  -H 'Content-Type: application/json' \
  -d '{"f0":0.1,"f1":-0.2,"f2":0.3,"f3":0.0,
       "f4":0.1,"f5":0.2,"f6":0.0,"f7":0.1,
       "f8":0.2,"f9":0.0}'
python3 scripts/week3.py api-check
python3 scripts/week3.py sizes --kinds single cpu onnx

trivy image --scanners vuln --severity HIGH,CRITICAL -f json \
  -o docs/week3/trivy-cpu.json mlops-cpu:local
trivy image --scanners vuln --severity HIGH,CRITICAL -f json \
  -o docs/week3/trivy-onnx.json mlops-onnx:local
trivy image -f spdx-json -o docs/week3/sbom-cpu.json mlops-cpu:local
python3 scripts/week3.py scan-report --gate
```

Laukiama: abi API `healthy`, UID ne 0, `api-check` — PASS, saugumo vartas — PASS. `start` paleidžia `docker run -d`, laukia parengties iki 90 s ir išsaugo matavimą. `api-check` tikrina prognozių sutapimą, JSON laukų tvarką ir blogų užklausų HTTP 422; papildomai kartoti tų pačių patikrų nereikia.

CPU ir mokymas naudoja suderintas sklearn versijas, ONNX gauna tik jam reikalingą runtime. CPU virtuali aplinka `/opt/venv` prieinama ne-root vartotojui.

Dydžiai: `docs/image-sizes.md`; skenai ir SBOM: `docs/week3/`; saugumo išvada: `docs/security-scan.md`.

6.0.3 2. Publikavimas į GHCR

GHCR reikia `write:packages` ir `read:packages` teisių. Toliau pateikta `gh auth login` komanda jų paprašo; prisijungimą užbaikite naršyklėje. Naujo SSH rakto kelti nereikia (`Skip`). Pasirinkite dar nenaudotą `VERSION`. Tokenas perduodamas tiesiai Docker, jo į failus ar pokalbį nekopijuokite.

```
#!/usr/bin/env bash
# Publish the local CPU/ONNX images after their scans pass.
# Run from tutorial/fraud-detection; authenticate interactively, never paste tokens.
set -eu
gh auth login --hostname github.com --git-protocol ssh --web \
  --scopes write:packages,read:packages
OWNER=$(gh api user --jq '.login | ascii_downcase')
gh auth token | docker login ghcr.io -u "$OWNER" --password-stdin
VERSION=v1.2.2-cpu
python3 scripts/week3.py scan-report --kinds cpu onnx --gate
docker tag mlops-cpu:local "ghcr.io/$OWNER/week3-cpu:$VERSION"
docker push "ghcr.io/$OWNER/week3-cpu:$VERSION"
docker tag mlops-onnx:local "ghcr.io/$OWNER/week3-onnx:$VERSION"
docker push "ghcr.io/$OWNER/week3-onnx:$VERSION"
```

Abu `docker push` turi baigtis sėkmingai ir išvesti digest. Ši seka publikuoja vietinę architektūrą. CRITICAL radinys sustabdo publikavimą; HIGH radinius aptarkite ataskaitoje.

6.0.4 3. Išsaugoti darbą

Vykdymo kodas jau yra Git. Į `docs/requirements.md` įrašykite šio darbo NFR ir išvadas: NFR-11 — CPU rootfs tikslas ≤500 MB, ONNX ≤300 MB; NFR-12 — ONNX parengtis ≤5 s su vietiniu vaizdu; NFR-13 — 0 CRITICAL prieš publikavimą; NFR-14 — API vartotojo UID ne 0. Dydžius, laiką ir neatitiktis pagrįskite sugeneruotais matavimais.

Įtraukite savo pakeitimus ir rezultatus; prieš commit peržiūrėkite `git diff --cached`, kad nepatektų kitos užduoties failai. `origin` turi rodyti jūsų repo:

```
git remote -v
git add .dockerignore docker/ src/ scripts/ security/ docs/ \
  constraints-week3.txt requirements-*.txt
git diff --cached
git commit -m "feat: Week 3 containers"
git push -u origin HEAD
```

Jei Git praneša `nothing to commit`, darbas jau išsaugotas. Baigus demonstraciją pašalinkite tik šios laboratorijos CPU/ONNX konteinerius:

```
python3 scripts/week3.py cleanup --kinds cpu onnx
```

6.0.5 GPU užduotis: tik NVIDIA Linux serveryje

Šiame minimaliame studentų rinkinyje GPU failų nėra. Šį skyrių vykdykite tik dėstytojui pateikus `docker/Dockerfile.gpu`, `src/serve_gpu.py` ir NVIDIA Linux serverį su tvarkykle bei Container Toolkit. Iki tol skyrių praleiskite; CPU/ONNX praktika nuo jo nepriklauso. Gavę šią aplinką, projekto kataloge vykdykite:

```
docker build --platform linux/amd64 \
  -f docker/Dockerfile.gpu -t mlops-gpu:local .
docker run --rm --gpus all mlops-gpu:local \
  python /app/src/serve_gpu.py --check
python3 scripts/week3.py start gpu --port 18082
```

Laukiama tikro CUDA skaičiavimo, `healthy` ir UID ne 0. Apple Silicon CUDA nepalaiko. Šis bandymas tikrina CUDA aplinką, ne sklearn modelio mokymą GPU. Papildomus GPU matavimus ir atsiskaitymą suderinkite su dėstytoju.

6.0.6 Papildomos užduotys

Šios savarankiškos užduotys išplečia pagrindinį pratimą:

1. **3.1. Dydžiai.** Palyginkite single, CPU ir ONNX. Paaiškinkite, kokių paketų ir failų runtime nebereikia.
2. **3.2. HEALTHCHECK.** Parodykite `unhealthy`; aptarkite skirtumą nuo HTTP 422.
3. **3.3. Multi-arch.** Sukurkite amd64/arm64 OCI arba registry manifestą.
4. **3.4. CI (bonus).** Automatizuokite build, API testus, skenavimą ir publikavimą. **Šiame rinkinyje CI workflow nėra**: pirmiausia jį parenkite arba gaukite iš dėstytojo. Vien Git žymos išsiuntimas CI nesukuria. Rankiniam GHCR pratimui CI nereikia.

Ketvirtos savaitės infrastruktūros medžiaga bus pateikta atskirai. Šiam darbui Terraform ir GCP išteklių nereikia.

6.0.7 Atsiskaitymas (iki 0,2 balo)

Užduoties rezultatas (deliverable)

D1: Dockerfile ir priklausomybės. D2: GHCR adresai bei digest, Git commit. D3: tikri Trivy JSON, SBOM ir saugumo išvada. D4: API ir sveikatos patikros; GPU atskirai. D5: NFR-11–NFR-14 faktai. D6: dydžių palyginimas su platforma.

Kriterijus	Balai	Įrodymas
CPU multi-stage ir ne-root	0–0,04	API healthy, UID ne 0
GPU aplinka	0–0,03	Tikras CUDA bandymas
ONNX eksportas ir API	0–0,04	Prognozių atitiktis
GHCR publikavimas	0–0,03	Vaizdų adresai ir digest
Trivy ir SBOM	0–0,03	Tikri radiniai, 0 CRITICAL vartas
NFR v1.2	0–0,02	Reikalavimai susieti su faktais
Demo ir paaiškinimas	0–0,01	Gyva patikra ir argumentai

CPU rootfs tikslas $\leq$500 MB, ONNX $\leq$300 MB; ONNX parengtis $\leq$5 s, kai vaizdas jau vietinis. Nepasiektą tikslą paaiškinkite; vienas matavimas nėra SLA. Sintetinė užduotis neįrodo realaus fraud modelio kokybės. Nesuteikus GPU aplinkos, jos vertinimą suderinkite su dėstytoju.

Klausimai: kodėl pavojingas `COPY .env`? Kuo skiriasi `.dockerignore` ir `.gitignore`? Kuris vaizdas didžiausias? Kaip vertinate HIGH CVE? Kodėl build neįrodo GPU veikimo?

7 Savarankiško darbo valandų suvestinė

Savaitė	Tema	Val.
1	MLOps įvadas, publikacijos skaitymas	5
2	Eksperimentų sekimo pratęsimas	5
3	Docker optimizavimas	6
4	Terraform, kaštų analizė	6
5	K8s operatoriai, training jobs	6
6	CI/CD tobulinimas	5
7	Inferencijos optimizavimas	6
8	GPU profiliavimas	5
9	Pipeline orkestravimas	6
10	Agentų tobulinimas	5
11	Observability stack	6
12	Scaling ir FinOps	5
13	Saugumo auditas	6
14	Edge ir FL	5
15	Testavimas, ataskaita	6
16	Egzamino pasiruošimas	5
Iš viso		**82**

8 Atsiskaitymo kalendorius

Savaitė	**Įvykis**	**Pastabos**
1–15	Individualių užduočių atsiskaitymai	Per laboratorinius darbus
8	Tarpinis atsiskaitymas	Individualios užduoties statusas (rekomenduojama $\geq$50 %)
16	Komandinio projekto gynimas	Privaloma egzaminui
Sesija	Rašytinis egzaminas	Atviri ir kompleksiniai klausimai

A Ubuntu paruošimas

Vienkartinis paruošimas **Ubuntu 22.04 arba 24.04** kompiuteryje ar asmeninėje VM. Reikia `systemd`, interneto ir `sudo` teisių. Bendrą serverį paruošia administratorius.

Šio skyriaus komandų Mac ar paprasto Docker konteinerio viduje nevykdykite. Jei visi įrankiai jau veikia, grįžkite į 3 savaitės praktiką.

Komandas kopijuokite į Bash kaip įprastas vartotojas. Diegti galima iš bet kurio katalogo. `sudo` prašo jūsų Ubuntu slaptažodžio; vedami simboliai terminale nerodomi.

A.1 Bendri įrankiai

```
bash
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg git python3
```

A.2 Docker Engine ir Buildx

Jei `docker info` ir `docker buildx version` jau veikia be `sudo`, šį poskyrį praleiskite. Toliau — švarios Ubuntu aplinkos diegimas iš oficialios Docker APT saugyklos. Jei turite kitą Docker diegimą ar paketų konfliktą, pirmiausia vadovaukitės tos instrukcijos konfliktuojančių paketų skyriumi; bendro serverio paketų nešalinkite.

Bloką nuo `sudo tee` iki atskiros `EOF` eilutės kopijuokite visą: tarp jų yra konfigūracijos tekstas. Pasvirusiuoju brūkšniu \ pažymėtos eilutės taip pat tęsiamos.

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
. /etc/os-release
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: ${VERSION_CODENAME}
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo groupadd -f docker
sudo usermod -aG docker "$(id -un)"
```

Visiškai atsijunkite nuo Ubuntu ir prisijunkite iš naujo. SSH atveju uždarykite ryšį ir sukurkite naują. Vien naujo terminalo lango gali nepakakti. Narystė `docker` grupėje suteikia root lygio prieigą, todėl bendrame serveryje prieigą nustato administratorius. Po prisijungimo patikrinkite be `sudo`:

```
docker info
docker buildx version
docker run --rm hello-world
```

Laukiama `Hello from Docker!`. Jei serveris nepasiekiamas ar neleidžiama naudoti socket, patikrinkite tarnybą ir savo narystę grupėje. Tik tada tęskite.

A.3 Trivy

Trivy atlieka Docker vaizdų pažeidžiamumų patikrą ir generuoja SBOM. Jei `trivy --version` jau veikia, šį poskyrį praleiskite. Naudojama oficiali Trivy APT saugykla:

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://aquasecurity.github.io/trivy-repo/deb/public.key \
  | sudo gpg --batch --yes --dearmor \
      -o /etc/apt/keyrings/trivy.gpg
sudo chmod a+r /etc/apt/keyrings/trivy.gpg
sudo tee /etc/apt/sources.list.d/trivy.sources >/dev/null <<EOF
Types: deb
URIs: https://aquasecurity.github.io/trivy-repo/deb
Suites: generic
Components: main
Signed-By: /etc/apt/keyrings/trivy.gpg
EOF
sudo apt-get update
sudo apt-get install -y trivy
trivy --version
```

A.4 GitHub CLI (gh)

GitHub CLI naudojamas prisijungti prie GitHub ir GHCR. Naršyklės prisijungimas jo neatstoja. Jei `gh --version` jau veikia, diegimą praleiskite. Naudojama oficiali GitHub CLI APT saugykla:

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL \
  https://cli.github.com/packages/githubcli-archive-keyring.gpg \
  -o /etc/apt/keyrings/githubcli-archive-keyring.gpg
sudo chmod a+r /etc/apt/keyrings/githubcli-archive-keyring.gpg
sudo tee /etc/apt/sources.list.d/github-cli.sources >/dev/null <<EOF
Types: deb
URIs: https://cli.github.com/packages
Suites: stable
Components: main
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/githubcli-archive-keyring.gpg
EOF
sudo apt-get update
sudo apt-get install -y gh
gh --version
```

A.5 Grįžimas į praktiką

Įrankiai įdiegti. Grįžkite į savo Git repo, atverkite 3 savaitės projekto katalogą ir tęskite nuo pradinės patikros. GitHub autorizacija atliekama praktikos GHCR skyriuje; prisijungimo žetonų į failus nerašykite.

Jei po diegimo komanda nerandama, diegimo klaidą išspręskite prieš tęsdami. Jei `sudo` teisės nesuteiktos, kreipkitės į aplinką paruošiantį administratorių.